

Adversarial Evasion Methodologies for Static Portable Executable Malware Detectors: Technical Analysis and Implementation Strategies for the PEGuard Framework

The defensive architecture of modern computing environments increasingly relies on machine learning models to identify and mitigate malicious software before execution. Traditional signature-based detection, while historically significant, has proven largely ineffective against the rapid proliferation of polymorphic and metamorphic malware strains.¹ In response, the industry has transitioned toward static analysis frameworks that leverage the structural and statistical properties of the Portable Executable (PE) file format. These models, such as the Convolutional Neural Network-based MalConv and the gradient-boosted decision tree-based Ember, extract high-dimensional feature vectors from headers, section tables, and raw byte distributions to classify files as either benign or malicious.⁴

However, the inherent susceptibility of machine learning models to adversarial examples presents a critical challenge for detection systems like PEGuard. Adversarial examples are malicious inputs that have been subjected to minimal, functionality-preserving perturbations designed to induce misclassification by the target model.⁷ For an adversarial attack to be considered successful in the malware domain, it must satisfy three rigorous constraints: it must be format-preserving, ensuring the binary adheres to the PE specification; it must be executable-preserving, allowing the file to load and run without error; and it must be maliciousness-preserving, maintaining the original harmful intent of the software.⁴ This report identifies and details four testable methods for modifying PE files to achieve evasion while strictly adhering to these constraints.

The Theoretical Framework of Static PE Feature Analysis

To understand the mechanisms of adversarial evasion, one must first analyze the feature space targeted by detectors. Static detectors typically process the PE file in two ways: raw-byte analysis and engineered feature analysis. Raw-byte models like MalConv utilize an embedding layer to map every possible byte value (0–255) into a vector space, followed by convolutional layers that learn local patterns indicative of malicious intent.⁴ Conversely, engineered models like Ember utilize the Library to Instrument Executable Formats (LIEF) to parse the file and

extract structured data.⁶

The Ember feature set consists of 2,381 distinct features categorized into several logical groups. These features provide a comprehensive view of the file's static properties, from its global statistics to its internal organizational structure.⁶

Feature Category	Dimension	Description
Byte Histogram	256	Normalized frequency of each byte value (0-255) in the file.
Byte-Entropy Histogram	256	Measures the local "randomness" of bytes throughout the file buffer.
String Information	104	Statistics on printable strings, including count, average length, and entropy.
General File Info	10	High-level metadata: file size, number of symbols, and timestamps.
Header Information	62	Fields from the COFF and Optional headers, such as machine type and characteristics.
Section Information	255	Metadata for each section: names, virtual sizes, entropy, and permissions.

Imports	1280	Counts and hashes of imported DLLs and specific function names.
Exports	128	Information regarding the functions exported by the binary.
Data Directories	30	Presence and sizes of various data directories (e.g., resources, relocations).

Detectors like those in the PEGuard project are trained to find the decision boundaries within this high-dimensional space. Adversaries exploit these boundaries by shifting a malicious file's feature vector across the threshold into the "benign" region.⁵

Method 1: Benign-Mimetic Section Addition and Statistical Distribution Blending

The most robust method for evading both raw-byte and feature-engineered models involves the injection of new sections containing content specifically crafted to mirror the statistical profile of legitimate software. This method targets the Byte Histogram and Byte Entropy features, which often serve as primary indicators of maliciousness in static models.⁶

Mechanism of Statistical Neutralization

Malicious files frequently exhibit highly skewed byte distributions. For instance, ransomware or packed trojans often possess high global entropy due to encrypted payloads or compressed code. Shannon entropy, a fundamental measure in these models, is calculated as follows:

$$H(X) = - \sum_{i=0}^{255} P(x_i) \log_2 P(x_i)$$

Where $P(x_i)$ is the probability of a specific byte value x_i occurring in the file.¹⁵ Legitimate software typically exhibits lower entropy in sections containing resources or strings, and the byte histogram often shows a higher frequency of common ASCII characters and null bytes. By

adding a new section—a structural unit of the PE format—an adversary can inject a large volume of "benign noise" that dilutes the malicious features without interfering with the existing execution logic.¹³

Research has demonstrated that "String Blending" is particularly effective. By extracting large blocks of text from a benign Microsoft binary, such as a license agreement or a resource-heavy utility, and placing them into a new section, the adversary forces the model to re-evaluate the file's global statistics.¹³ Furthermore, adding a section filled with a constant byte value—termed the "Constant Byte Attack"—has shown remarkable success against neural network models like MalConv. For example, injecting 100,000 bytes of the value 0xA9 was found to cause a catastrophic drop in detection rates in some studies.¹³

Implementation Workflow using LIEF

The implementation of this method requires careful management of the PE section table and alignment properties. The LIEF library simplifies this by automating the calculation of Relative Virtual Addresses (RVAs) and rebuilding the necessary headers.¹⁶

1. **Alignment Retrieval:** The binary is parsed to identify the `file_alignment` and `section_alignment` values from the Optional Header. New sections must have a `SizeOfRawData` that is a multiple of the `file_alignment`.¹⁷
2. **Section Generation:** A new `lief.PE.Section` object is created. The content is populated with either benign strings or optimized constant bytes.
3. **Characteristics Configuration:** The section's characteristics are set to `CNT_INITIALIZED_DATA` and `MEM_READ`. This ensures the section is loaded into memory but is not treated as executable code, which minimizes the risk of triggering behavioral alerts.¹⁷
4. **Integration:** The section is added to the binary using `pe_file.add_section(new_section)`. LIEF automatically updates the `NumberOfSections` field in the COFF header and appends the new entry to the section table.¹⁶
5. **Rebuilding:** The final binary is written to disk. LIEF recomputes the `SizeOfImage` in the Optional Header to account for the new memory requirements of the injected section.¹⁶

Predicted Model Impact

The impact of section addition is observed in the SHAP (SHapley Additive exPlanations) values of the target model. SHAP analysis reveals that for gradient-boosted models, the injection of benign strings shifts the decision from malicious branches toward benign ones by de-emphasizing the importance of encrypted sections.⁴

Metric	Original Malware	Modified	Model Change

		(Adversarial)	
Global Entropy	7.8 (High)	5.2 (Moderate)	Benign Shift
Section Count	4	5	Structural Alert
Byte 0x00 Freq	Low	High (due to padding)	Benign Shift
MalConv Score	0.999	0.012	Evasion

While this method is highly effective, it increases the total file size. However, given that many legitimate applications are large, a size increase of several hundred kilobytes is rarely a reliable indicator of maliciousness.¹

Method 2: Semantic Import Obfuscation and Benign API Blending

The second method focuses on the Import Address Table (IAT) and the Import Lookup Table (ILT). Static detectors extract features from these tables to build a behavioral profile based on the external functions a program calls.²⁰ Malicious files often import functions related to process injection, registry modification, or network communication. This method involves the injection of multiple benign imports to mask these malicious signals.²⁰

Mechanism of Feature Collision and Dilution

Detection models often utilize feature hashing to convert imported DLL names and function strings into a fixed-length vector.⁶ If a file imports `kernel32.dll!VirtualAllocEx` and `kernel32.dll!WriteProcessMemory`, it creates a strong feature cluster associated with process hollowing or injection.²¹ The goal of semantic obfuscation is to add a high volume of benign imports—such as those from `GDI32.dll`, `user32.dll`, or various C runtime libraries—so that the hashed feature vector is dominated by benign indicators.²⁰

This attack exploits the fact that many ML models treat imports as a "bag of words" rather than a sequential sequence. By increasing the number of unique benign imports, the adversary

lowers the relative importance of the malicious imports within the model's decision logic.⁵ This is particularly effective against LightGBM models trained on Ember, where the import features are highly influential.⁴

Implementation Strategy for Import Injection

Manipulating the import table is technically challenging because it involves updating several interconnected data structures, including the IMAGE_IMPORT_DESCRIPTOR array and the IAT itself.²⁰

1. **Target API Selection:** Identify common benign DLLs found in legitimate system tools, such as api-ms-win-crt-stdio-l1-1-0.dll or advapi32.dll. Select functions that are pervasive but non-threatening, such as puts, time, or GetSystemTime.²⁰
2. **API Augmentation:** Using LIEF's add_import method, new libraries and entries are added. For instance: `pe.add_import("kernel32.dll").add_entry("GetStartupInfoW")`.²⁰
3. **Table Relocation:** Because the original import table is usually located in a fixed-size region, LIEF handles the complexity of creating a new, larger import table in a new section (often named .I1 or similar) and updating the Data Directory pointer.²²
4. **Trampoline Patching:** To maintain functionality, the original calls within the .text section must still resolve correctly. LIEF ensures the new IAT preserves the original function addresses while adding the new, non-functional benign entries.²⁰
5. **Builder Configuration:** The modification must be finalized with `lief.PE.Builder.config_t.imports = True` to ensure the library re-calculates the IAT offsets and serializes the new table correctly.¹⁶

Predicted Model Impact and Profile Shifting

This method primarily affects the "Imports" and "Data Directories" feature groups in the Ember model. By "padding" the import list, the adversary creates a scenario where the malicious APIs are outliers in a predominantly benign set.

Model Feature	Malicious Profile	Augmented Profile	Decision Impact
Import Count	12	150	Benign Bias
Unique DLLs	2	15	Benign Bias

Hashed Import Vector	Malicious Centroid	Benign Centroid	Evasion
SHAP Importance	High (Injection APIs)	Low (Diluted)	Evasion

Research has indicated that while adversarial training can help models recognize the "fingerprint" of a relocated import table (such as the presence of a new section named .I1), the underlying technique of API blending remains a significant hurdle for static analysis.⁵

Method 3: Header Metadata Normalization and Artifact Modification

The third method involves the "surgical" modification of metadata fields within the PE headers. Machine learning models often pick up on subtle artifacts or "outliers" in the headers that are indicative of malware development environments or evasion tools.² By normalizing these fields to match the values found in common, professionally developed benign binaries, the malicious sample is disguised as a legitimate system file.⁶

Mechanism of Artifact Fingerprint Removal

PE headers contain numerous fields that are not strictly necessary for the execution of a user-mode program but are highly useful for classification. These include:

- **TimeStamp**: Many malware authors set this to zero or to an obviously faked date. Benign files almost always have a stamp corresponding to their compilation date.⁶
- **MajorOperatingSystemVersion**: Malware often retains default linker values (e.g., 4.0), whereas modern Windows 10/11 applications typically have values of 6.0 or 10.0.²⁹
- **Checksum**: While the Windows loader typically ignores this field for most EXE files, many ML models use a mismatched or zeroed checksum as a feature indicating tampering.⁶
- **Rich Header**: A hidden structure found in binaries compiled with Microsoft Visual Studio. It contains information about the build environment. Malware compiled with non-standard tools often lacks this header, while those that do have it may contain suspicious entries.¹⁴

By normalizing these artifacts, an adversary can "calm" the model's heuristics. For example, replacing a suspicious timestamp with one taken from a legitimate benign sample (e.g., notepad.exe) can significantly lower the malicious score in some gradient-boosted models.⁶

Implementation via Header Manipulation

Header modification is efficient and minimizes the risk of breaking the file, as it changes only a few dozen bytes.²⁶

- 1. **Timestamp Adjustment:** Access the file_header and set the timedatestamp to a realistic value. For instance: pe.header.timedatestamp = 1600000000.
- 2. **Version Normalization:** Update the MajorOperatingSystemVersion, MajorImageVersion, and MajorSubsystemVersion in the Optional Header to 10, reflecting a modern build environment.³⁴
- 3. **Checksum Recomputation:** Use LIEF to re-calculate the correct checksum: pe.optional_header.checksum = pe.optional_header.computed_checksum().³²
- 4. **Rich Header Handling:** LIEF allows for the removal or re-computation of the Rich Header. Removing it can eliminate tool-specific signatures, while re-building it can simulate a standard Visual Studio environment.²⁹
- 5. **Characteristics Modification:** The Characteristics flags in the File Header are analyzed. Malicious files often have flags indicating they are "stripped" or lack a relocation table. These can be modified to match benign profiles (e.g., enabling IMAGE_FILE_EXECUTABLE_IMAGE and IMAGE_FILE_32BIT_MACHINE).²⁷

Predicted Model Impact

The impact is most significant on models that emphasize "General File Information" and "Header Information." In the Ember dataset, these categories contain several high-importance features for identifying legacy malware or those produced by specific "builders".⁶

Header Field	Suspicious State	Normalized State	Impact
Timestamp	0 (1970)	1711843200 (2024)	Benign Shift
Checksum	0x00000000	0x0002A1B5 (Correct)	Integrity Check
OS Version	4.0	10.0	Environment Shift
Rich Header	Missing	Valid Build Artifacts	Fingerprint Removal

This method is highly efficient and leaves the functional code completely untouched. When combined with other techniques, it ensures the file does not trigger "obvious" heuristic alarms before the model begins deeper analysis of the sections and payloads.⁶

Method 4: Problem-Space Perturbations: Intra-section Code Caves and Adaptive Overlays

The fourth method utilizes "slack regions" and the file overlay to inject adversarial perturbations that do not affect the binary's execution but fundamentally alter the way the model perceives the raw byte stream. This technique is particularly effective against MalConv and other CNN-based detectors that process the file as a contiguous sequence of bytes.¹²

Mechanism of Slack Space and Overlay Injection

Modern PE files are subject to alignment requirements on disk and in memory. This often creates "slack space" or "code caves"—empty, unused regions between the end of the actual data in a section and the section's aligned boundary.¹³ Additionally, the "Overlay" is the area of a file that follows the last section defined in the section table. The Windows loader ignores the overlay, meaning any data placed there will never be executed or mapped into memory.¹³

Adversaries use these regions to store "adversarial noise." Unlike random noise, these perturbations are often calculated using gradient-based optimization methods such as the Fast Gradient Sign Method (FGSM). The optimization objective is to find a small perturbation δ such that the classifier $C(x + \delta)$ labels the malicious sample x as benign, while ensuring that δ resides entirely within non-functional regions like the overlay or slack space.⁴

Implementation of Non-Functional Perturbations

1. **Overlay Appending:** This is the simplest approach. An "adversarial payload"—potentially a large block of bytes optimized to lower the model's score—is appended to the end of the file. Because it is in the overlay, it preserves 100% functional integrity.¹³
2. **Slack Space Identification:** The researcher analyzes the discrepancy between each section's SizeOfRawData and its actual content. Any null-filled region at the end of a section is a target for perturbation injection.²⁶
3. **Intra-section Code Caves:** A more complex approach involves dynamically creating space within a section by shifting its contents and updating internal pointers. This "noble way" of manipulation allows the adversary to place perturbations in the middle of the file, where they are more likely to interfere with the convolutional filters of a model like MalConv.²⁶
4. **Metamorphic Restoration:** If perturbations are placed in a section that is mapped to

memory, a small "loader" or "stub" can be added to the entry point. Upon execution, the stub overwrites the adversarial perturbations with the original malicious code before running the payload. This ensures that the model sees the "benign" version of the file, but the OS executes the "malicious" version.²⁶

Predicted Model Impact and Evasion Efficacy

This method directly attacks the convolutional layers of neural networks. By placing perturbations in regions that the model treats as relevant but the program treats as empty, the adversary exploits a fundamental mismatch between the "model's view" and the "OS view" of the file.¹²

Modification Strategy	Location	Success Rate (MalConv)	Complexity
Append Attack	Overlay	16-25%	Low
Slack Padding	End of Sections	30-50%	Medium
Intra-section FGSM	Inside Sections	92-96%	High
Header Perturbation	Optional Header	10-15%	Low

Research indicates that intra-section code caves are significantly more effective than overlay padding, as models have become increasingly robust against data appended to the end of files.³⁷

Functional Integrity Verification and Evasion Validation

A critical requirement of the PEGuard framework is the preservation of malware functionality. Adversarial modifications often involve complex structural changes that can inadvertently break the binary.⁷ To ensure the modified samples remain operational, a multi-stage verification process is necessary.

Structural and Dynamic Verification

1. **Format Validation:** Following modification with LIEF, the binary must be checked against the PE specification. Tools like pefile or LIEF's own parser can verify that the headers are consistent and the section table is valid.¹⁴
2. **Static Control Flow Comparison:** A robust method involves comparing the Control Flow Graph (CFG) of the original malware with the modified version. If the CFG—the map of all possible execution paths—remains identical, the modification is considered functionally successful at the static level.⁷
3. **Dynamic Execution (Sandboxing):** The ultimate test of functionality is execution in an isolated environment such as the Cuckoo Sandbox. This process monitors API calls, network traffic, and file system changes. If the adversarial example exhibits the same behavioral signatures as the original malware, it is confirmed to be "maliciousness-preserving".⁷
4. **Adversarial Scoring:** Finally, the modified file is passed to the target ML model (e.g., Ember/LightGBM). Evasion is confirmed if the model's output score drops below the detection threshold (e.g., $P(\textit{malicious}) < 0.5$).¹

Comparative Analysis of Evasion Strategies

The following table synthesizes the four methods, evaluating their complexity, risk to functionality, and predicted impact on contemporary static detectors.

Method	Primary Target	Technical Complexity	Functional Risk	Implementation Tool
Section Blending	Global Stats	Moderate	Low	LIEF add_section
Import Masking	Behavioral Profile	High	Moderate	LIEF add_import
Header Normalization	Heuristic Artifacts	Low	Very Low	LIEF header props

Code Cave/Overlay	Raw Byte Stream	High	High (Caves)	LIEF + Manual Calc
-------------------	-----------------	------	--------------	--------------------

The analysis suggests that for the PEGuard project, a "hybrid" approach is most effective. Combining Header Normalization with Section Blending addresses both high-level semantic features and low-level byte distributions, providing the highest probability of evasion across multiple model architectures.⁴

Strategic Implications for the PEGuard Detection Framework

The identification of these four methods highlights the inherent vulnerability of static analysis to problem-space adversarial attacks. Machine learning models, while powerful, often rely on discriminative features that are not causally linked to malicious behavior. An adversary can manipulate these "secondary" features—such as timestamps, section names, or padding bytes—to flip a model's decision without ever touching the primary malicious logic.⁵

To build a more robust detection system, PEGuard must incorporate defensive measures such as "Adversarial Training." This involves generating evasive samples using the methods described in this report and including them in the model's training dataset. Research shows that training a model on adversarial examples produced by LIEF or reinforcement learning agents can reduce the effectiveness of future attacks by up to 33%.⁴ Furthermore, the transition toward models that utilize more stable structural features—such as those derived from the program's control flow—may offer greater resilience against simple header and section manipulations.¹

In conclusion, the four methods detailed—Benign-Mimetic Section Addition, Semantic Import Obfuscation, Metadata Normalization, and Adaptive Perturbation Injection—provide a comprehensive and testable set of strategies for the PEGuard project. These methods exploit the discrepancy between how machine learning models perceive binary files and how the Windows operating system executes them, offering a critical pathway for evaluating and improving the robustness of modern malware detection systems.

Works cited

1. An Adversarial Attack on ML-Based IoT Malware Detection Using Binary Diversification Techniques - IEEE Xplore, accessed March 31, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10380310/10786223.pdf>
2. Static Analysis and Machine Learning-based Malware Detection System using PE Header Feature Values - Semantic Scholar, accessed March 31, 2026, <https://pdfs.semanticscholar.org/e301/fe2677c4b6bc3ab6f53b18da13737bbc6d3e.pdf>

3. Dynamic Malware Classification of Windows PE Files using CNNs and Greyscale Images Derived from Runtime API Call Argument Conversion - arXiv, accessed March 31, 2026, <https://arxiv.org/html/2505.24231v1>
4. Evaluating Realistic Adversarial Attacks against Machine Learning ..., accessed March 31, 2026, <https://www.mdpi.com/1999-5903/16/5/168>
5. Learning to Evade Static PE Machine Learning Malware ... - arXiv, accessed March 31, 2026, <https://arxiv.org/pdf/1801.08917>
6. Evading Static Machine Learning Malware Detection Models – Part 2: The Gray-Box Approach - Compass Security Blog, accessed March 31, 2026, <https://blog.compass-security.com/2020/11/evading-static-machine-learning-malware-detection-models-part-2-the-gray-box-approach/>
7. Hard-Label Adversarial Attack Against Static Malware Detection with Minimal Perturbation, accessed March 31, 2026, <https://www.ijcai.org/proceedings/2025/622>
8. Practical Attacks on Machine Learning: A Case Study on Adversarial Windows Malware, accessed March 31, 2026, <https://www.computer.org/csdl/magazine/sp/2022/05/09817418/1EOAbnvdKdy>
9. Adversarial Attacks against Windows PE Malware Detection: A Survey of the State-of-the-Art - arXiv, accessed March 31, 2026, <https://arxiv.org/pdf/2112.12310>
10. A Survey on Adversarial Attacks for Malware Analysis - IEEE Xplore, accessed March 31, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10820123/10806701.pdf>
11. Adversarial Binaries: AI-guided Instrumentation Methods for Malware Detection Evasion - OSTI, accessed March 31, 2026, <https://www.osti.gov/pages/servlets/purl/2587493>
12. Explainability Guided Adversarial Evasion Attacks on Malware Detectors - arXiv, accessed March 31, 2026, <https://arxiv.org/html/2405.01728v1>
13. Evading Machine Learning Malware Classifiers | by William Fleshman - Medium, accessed March 31, 2026, <https://medium.com/data-science/evading-machine-learning-malware-classifiers-ce52dabdb713>
14. EMBER2024: Advancing Cybersecurity ML Training on Evasive Malware - CrowdStrike, accessed March 31, 2026, <https://www.crowdstrike.com/en-us/blog/ember-2024-advancing-cybersecurity-ml-training-on-evasive-malware/>
15. Analyzing and comparing the effectiveness of malware detection: A study of machine learning approaches - PMC, accessed March 31, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10770453/>
16. PE — LIEF Documentation, accessed March 31, 2026, <https://lief.re/doc/stable/formats/pe/index.html>
17. Writing a PE packer – Part 3 : packing with python - WireDiver, accessed March 31, 2026, <https://wirediver.com/tutorial-writing-a-pe-packer-part-3/>
18. Creating Valid Adversarial Examples of Malware - arXiv, accessed March 31, 2026, <https://arxiv.org/html/2306.13587>
19. An Adversarial Attack on ML-Based IoT Malware Detection Using Binary Diversification Techniques, accessed March 31, 2026,

- https://faculty.csie.ntust.edu.tw/~smcheng/papers/J41_2412_An%20Adversarial%20Attack%20on%20ML-based%20IoT%20Malware%20Detection%20Using%20Binary%20Diversification%20Techniques.pdf
20. Imports Modification — LIEF Documentation, accessed March 31, 2026, <https://lief.re/doc/stable/formats/pe/modifications/imports.html>
 21. PE Import Analyzer: A Practical Guide for Malware Analysts and Reverse Engineers | by Andrey Pautov | Feb, 2026 | InfoSec Write-ups, accessed March 31, 2026, <https://infosecwriteups.com/pe-import-analyzer-a-practical-guide-for-malware-analysts-and-reverse-engineers-29b8b98aeaf3>
 22. Injecting Code using Imported Functions into Native PE Files - Washi, accessed March 31, 2026, <https://blog.washi.dev/posts/import-patching/>
 23. MITRE ATT&CK T1055.002 Process Injection: Portable Executable Injection - Picus Security, accessed March 31, 2026, <https://www.picussecurity.com/resource/blog/t1055-002-portable-executable-injection>
 24. Cannot add section and rebuild PE on Windows 10 · Issue #109 · lief-project/LIEF - GitHub, accessed March 31, 2026, <https://github.com/lief-project/LIEF/issues/109>
 25. Several issues around modifying and writing PE files using the Python API #1167 - GitHub, accessed March 31, 2026, <https://github.com/lief-project/LIEF/issues/1167>
 26. Exploiting Windows PE Structure for Adversarial ... - NSF PAR, accessed March 31, 2026, <https://par.nsf.gov/servlets/purl/10423459>
 27. Static Detection of Malware and Benign Executable Using Machine Learning Algorithm - UPV, accessed March 31, 2026, https://personales.upv.es/thinkmind/dl/conferences/internet/internet_2016/internet_2016_1_40_40038.pdf
 28. (PDF) Detecting Malware in Portable Executable Files using Machine Learning Approach, accessed March 31, 2026, https://www.researchgate.net/publication/367683560_Detecting_Malware_in_Portable_Executable_Files_using_Machine_Learning_Approach
 29. Static Detection of Portable Executable Files - winternl, accessed March 31, 2026, <https://winternl.com/static-detection-of-portable-executable-files/>
 30. Thread: What are the valid values for the OS Version fields in the PE header? - VBForums, accessed March 31, 2026, <https://www.vbforums.com/showthread.php?841507-What-are-the-valid-values-for-the-OS-Version-fields-in-the-PE-header>
 31. Analysing the Malware by using Checksum and Signature-Based Detection Techniques - GRENZE Scientific Society, accessed March 31, 2026, <https://thegrenze.com/pages/servej.php?fn=48.pdf&name=Analysing%20the%20Malware%20by%20using%20Checksum%20andSignature-Based%20Detection%20Techniques&id=1834&association=GRENZE&journal=GJET&year=2023&volume=9&issue=2>
 32. LIEF v0.12.0, accessed March 31, 2026, <https://lief.re/blog/2022-03-27-lief-v0-12-0/>
 33. Static Analysis Research - Windows PE - DEV Community, accessed March 31,

- 2026, <https://dev.to/syrull/static-analysis-research-windows-pe-f6b>
34. Library to Instrument Executable Formats: LIEF::PE::OptionalHeader Class Reference, accessed March 31, 2026, https://lief.re/doc/latest/doxygen/classLIEF_1_1PE_1_1OptionalHeader.html
 35. Python — LIEF Documentation, accessed March 31, 2026, <https://lief.re/doc/stable/formats/pe/python.html#optionalheader-class-reference>
 36. A Malware Detection Scheme Based on Mining Format Information - PMC, accessed March 31, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC4060536/>
 37. Intra-Section Code Cave Injection for Adversarial Evasion Attacks on Windows PE Malware File - arXiv, accessed March 31, 2026, <https://arxiv.org/pdf/2403.06428>
 38. Evading Machine Learning Malware Detection - Black Hat, accessed March 31, 2026, <https://blackhat.com/docs/us-17/thursday/us-17-Anderson-Bot-Vs-Bot-Evading-Machine-Learning-Malware-Detection-wp.pdf>
 39. Towards Feature Space Adversarial Attack by Style Perturbation, accessed March 31, 2026, <https://ojs.aaai.org/index.php/AAAI/article/view/17259/17066>
 40. Adversarial Training to Improve Accuracy and Robustness of a Windows PE Malware Detection Model - CEUR-WS.org, accessed March 31, 2026, <https://ceur-ws.org/Vol-3962/paper2.pdf>